

Navigators Technical Documentation

Navigators — Technical Approach Document

Document Identifier: TECH-NAV-01
Status: Production Reference
Version: 1.0.0
Last Updated: 2026-09-23
Attribution: Developed by Navigators

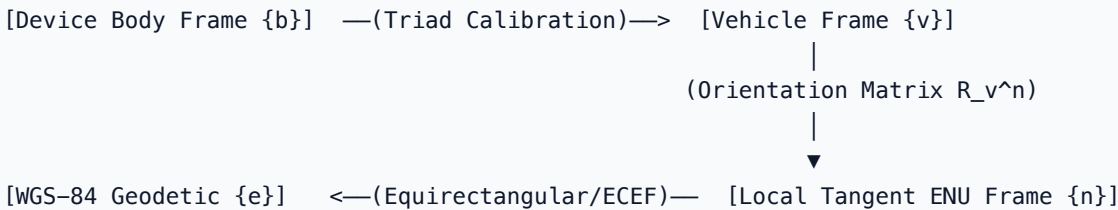
1. Sensor Acquisition & Sampling Characteristics

Navigators collects inertial and position data via standardized Web APIs executing within the smartphone browser runtime:

- **Linear Acceleration with Gravity:** Acquired via HTML5 `devicemotion` (`event.accelerationIncludingGravity`), reporting in meters per second squared (m/s^2) in the device coordinate frame.
- **Rotational Angular Rate:** Acquired via HTML5 `devicemotion` (`event.rotationRate`), reporting angular velocity in degrees per second ($^\circ/\text{s}$) or radians per second (rad/s) about device axes X, Y, Z .
- **GNSS Position & Kinematics:** Acquired via HTML5 Geolocation API (`navigator.geolocation.watchPosition`), providing WGS-84 latitude, longitude, ellipsoidal altitude, horizontal speed (m/s), ground track heading ($^\circ$), and 1-sigma horizontal accuracy radius (m).
- **Sampling Frequency:** Mobile browser sensor event dispatches are governed by the hardware clock and operating system scheduler, nominally operating between 10 Hz and 60 Hz. The system resamples incoming telemetry into a deterministic 10 Hz internal cycle ($\Delta t = 0.1 \text{ s}$).

2. Coordinate Frames & Transformations

Navigators explicitly manages four distinct reference frames to maintain mathematical consistency:



1. **Device Body Frame ($\{b\}$):** Origin fixed at device center-of-mass. X_b points toward the right edge of the screen, Y_b points toward the top edge, and Z_b points orthogonally out of the screen toward the user.
2. **Vehicle Body Frame ($\{v\}$):** Origin fixed at vehicle center-of-mass. X_v points transversely toward the passenger door (lateral), Y_v points along the longitudinal centerline forward (forward velocity), and Z_v points orthogonally upward through the vehicle roof.
3. **Local Tangent Navigation Frame ($\{n\}$ - ENU):** Cartesian local coordinate frame with origin established at the initial valid GNSS fix. X_n points true East, Y_n points true North, and Z_n points vertically Upward.

4. **Earth-Centered Earth-Fixed Frame ($\{e\}$ - WGS-84):** Global geodetic coordinates (ϕ, λ, h) representing latitude, longitude, and height.

3. Dynamic Phone-to-Vehicle Alignment (Triad Algorithm)

Because smartphones are mounted arbitrarily on dashboard clips, windshield suction brackets, or magnetic holders, the device body frame $\{b\}$ does not coincide with vehicle frame $\{v\}$. Navigators computes the constant rotation matrix $\mathbf{R}_b^v$ using a modified two-vector Triad algorithm during the initial movement phase:

1. **Vertical Leveling ($\mathbf{v}_1$):** During stationary dwell before driving begins, the only measured force is local gravity $\mathbf{g}_0 = [0, 0, 9.81]^T \text{ m/s}^2$. The normalized gravity vector measured in the phone frame is:

$$\mathbf{u}_{\text{down}}^b = \frac{\mathbf{a}_{\text{dwell}}^b}{\|\mathbf{a}_{\text{dwell}}^b\|}$$

2. **Longitudinal Alignment ($\mathbf{v}_2$):** When the vehicle accelerates forward from rest, the inertial accelerometer senses longitudinal acceleration $\mathbf{a}_{\text{forward}}$. Subtracting estimated gravity leaves the forward acceleration vector:

$$\mathbf{u}_{\text{fwd}}^b = \frac{\mathbf{a}_{\text{motion}}^b - (\mathbf{a}_{\text{motion}}^b \cdot \mathbf{u}_{\text{down}}^b) \mathbf{u}_{\text{down}}^b}{\|\mathbf{a}_{\text{motion}}^b - (\mathbf{a}_{\text{motion}}^b \cdot \mathbf{u}_{\text{down}}^b) \mathbf{u}_{\text{down}}^b\|}$$

3. **Triad Orthonormal Basis Construction:**

$$\mathbf{r}_1 = \mathbf{u}_{\text{fwd}}^b$$

$$\mathbf{r}_2 = \frac{\mathbf{u}_{\text{fwd}}^b \times \mathbf{u}_{\text{down}}^b}{\|\mathbf{u}_{\text{fwd}}^b \times \mathbf{u}_{\text{down}}^b\|}$$

$$\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2$$

$$\mathbf{R}_b^v = [\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3]^T$$

4. **Fallback PCA Alignment:** If forward acceleration is insufficiently distinct, the system accumulates a 10-second buffer of linear acceleration and computes the principal eigenvector via Principal Component Analysis (PCA) to extract the dominant motion axis.

4. Signal Processing & Causal Denoising

Low-cost smartphone IMUs suffer from engine vibration, chassis resonance, and road surface shocks. Navigators implements a two-stage real-time filter: 1. **Running Median Filter ($N = 5$):** A five-point sliding buffer removes unphysical acceleration spikes and sensor dropout glitches. 2. **First-Order Causal Low-Pass Filter ($f_c = 20 \text{ Hz}$):** Formulated in difference-equation format to guarantee zero future-sample leakage:

$$y[k] = \alpha x[k] + (1 - \alpha)y[k - 1]$$

where $\alpha = \frac{2\pi f_c \Delta t}{1 + 2\pi f_c \Delta t}$.

5. Feature Construction & Normalization

The AI velocity pipeline constructs rolling feature windows of 200 consecutive timesteps (20 s at 10 Hz):

$$\mathbf{X} \in \mathbb{R}^{200 \times 6} = [\mathbf{a}_x, \mathbf{a}_y, \mathbf{a}_z, \boldsymbol{\omega}_x, \boldsymbol{\omega}_y, \boldsymbol{\omega}_z]$$

Z-Score Normalization

Features are normalized using training statistics computed strictly over training sessions:

$$\hat{\mathbf{X}} = \frac{\mathbf{X} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

where $\boldsymbol{\mu} \in \mathbb{R}^6$ and $\boldsymbol{\sigma} \in \mathbb{R}^6$ are stored in `simulator/norm_stats.json`.

6. AI Velocity Estimation (TCN Architecture)

The core neural model is a **Temporal Convolutional Network (TCN)** engineered for edge inference: - **Causal**

Convolutions: Left-padded by $(K - 1) \cdot d$ samples so that output at step t is strictly independent of any $t' > t$.

- **Dilated Residual Blocks:** 4 blocks with exponentially increasing dilation factors $d \in \{1, 2, 4, 8\}$ and kernel size $K = 7$, yielding an effective receptive field spanning the full 200-sample window. - **Channel Progression:**

$[64, 64, 128, 128]$ feature channels with skip projections. - **Regression Head:** Global average pooling across time collapses the sequence to a 128-dimensional embedding, followed by a linear projection layer outputting 2D velocity $[v_N, v_E]$ in m/s. - **Edge Deployment:** Exported to ONNX with dynamic batch sizing and executed inside the browser via WebAssembly SIMD (`ort-wasm-simd-threaded.wasm`).

7. Extended Kalman Filter (EKF) Sensor Fusion

The navigation state is governed by a 15-state EKF formulated in the Local ENU frame:

$$\mathbf{x} = [\mathbf{p}_{ENU}^T, \mathbf{v}_{ENU}^T, \boldsymbol{\theta}^T, \mathbf{b}_a^T, \mathbf{b}_g^T]^T \in \mathbb{R}^{15}$$

Position (E, N, U)	[0:3]
Velocity (vE, vN, vU)	[3:6]
Attitude (roll, pitch, yaw)	[6:9]
Accelerometer Bias	[9:12]
Gyroscope Bias	[12:15]

Discrete Prediction Step ($\Delta t = 0.1$ s)

$$\mathbf{x}_{k|k-1} = \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{a}_{meas}, \boldsymbol{\omega}_{meas})$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^T + \mathbf{Q}_k$$

Multi-Rate Measurement Updates

The filter incorporates 4 distinct measurement updates as they become available: 1. **GNSS Position & Velocity Update** (when satellite lock is active):

$$\mathbf{z}_{gnss} = [\mathbf{p}_{gnss}^T, \mathbf{v}_{gnss}^T]^T, \quad \mathbf{R}_{gnss} = \text{diag}(\sigma_p^2, \sigma_p^2, \sigma_p^2, \sigma_v^2, \sigma_v^2, \sigma_v^2)$$

2. **AI Velocity Pseudo-Measurement Update** (during GNSS outage):

$$\mathbf{z}_{ai} = [v_E^{ai}, v_N^{ai}]^T, \quad \mathbf{R}_{ai} = \text{diag}(\sigma_{ai}^2, \sigma_{ai}^2)$$

3. Non-Holonomic Constraints (NHC): Assuming standard non-drifting wheeled vehicle dynamics, lateral velocity v_y^V and vertical velocity v_z^V in the vehicle frame are approximately zero:

$$\mathbf{z}_{\text{nhc}} = [0, 0]^T, \quad \mathbf{H}_{\text{nhc}} = \begin{bmatrix} -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{R}_{\text{nhc}} = \text{diag}(1.0, 1.0)$$

4. Zero-Velocity Update (ZUPT): When stationary dwell criteria are met, the filter applies full 3D velocity zeroing:

$$\mathbf{z}_{\text{zupt}} = [0, 0, 0]^T, \quad \mathbf{R}_{\text{zupt}} = \text{diag}(0.01, 0.01, 0.01)$$

Covariance Stability (Joseph Form)

To maintain positive-definiteness under single-precision floating-point arithmetic in WebAssembly, all measurement updates execute the Joseph stabilized form:

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}_{k|k-1}(\mathbf{I} - \mathbf{K}\mathbf{H})^T + \mathbf{K}\mathbf{R}\mathbf{K}^T$$

8. Anti-Jump GNSS Reacquisition

When exiting tunnels or underground areas, raw GNSS fixes frequently experience multipath transients and instant position jumps of 20–50 m. Direct filter assignment causes unacceptable visual and routing discontinuities. Navigators implements an anti-jump reacquisition governor: - When valid GNSS fixes resume after an outage, the system enters the **REACQUISITION** state. - Step-wise positional adjustments are bounded by:

$$\Delta \mathbf{p}_{\text{step}} = \min(\|\mathbf{p}_{\text{gnss}} - \mathbf{p}_{\text{est}}\|, \Delta p_{\text{max}}) \cdot \frac{\mathbf{p}_{\text{gnss}} - \mathbf{p}_{\text{est}}}{\|\mathbf{p}_{\text{gnss}} - \mathbf{p}_{\text{est}}\|}$$

where $\Delta p_{\text{max}} = 2.0$ m. - Over a 10-step horizon (1.0 s), the estimated trajectory converges smoothly onto the true satellite fix.

9. Geometric Vector Map Matching

Estimated ENU coordinates are projected onto local OpenStreetMap (OSM) road segment centerlines stored in client memory: 1. **Spatial Indexing:** Candidate road links within a 30 m search radius are retrieved from local storage. 2. **Orthogonal Projection:** The filter estimates the orthogonal foot point on each segment line:

$$t = \frac{(\mathbf{p} - \mathbf{a}) \cdot (\mathbf{b} - \mathbf{a})}{\|\mathbf{b} - \mathbf{a}\|^2}, \quad t \in [0, 1]$$

$$\mathbf{p}_{\text{proj}} = \mathbf{a} + t(\mathbf{b} - \mathbf{a})$$

3. Scoring Function: Segments are evaluated combining perpendicular distance and heading alignment:

$$S = d_{\perp} + w_{\theta} |\theta_{\text{vehicle}} - \theta_{\text{segment}}|$$

4. Snapping: If the best candidate score is below the rejection threshold, the UI render point snaps to the road centerline, preventing lateral drift accumulation during long straight road segments.

Developed by Navigators